

Sprint Plan

File Attachments on Reviews & Conclusions

Students and teachers can attach files to reviews and conclusions

User Stories: US1, US12 (extensions)

The Chipmunks — COSC 224 Winter 2026

Team Size: 6 Developers

Peer Evaluation Application

Feature B: File Attachments on Reviews & Conclusions

User Stories: US1 (Peer Review Submission) extension, US12 (Feedback Viewing) extension

Goal: Both students and teachers can attach files when submitting reviews or conclusions.

Dependency: This feature builds on Feature A's file upload infrastructure. Feature A's backend should merge first.

Overview

This feature extends the existing peer review submission flow (US1/US11) and feedback/conclusion workflow to support file attachments from both sides. Students can attach files (PDF, images, documents) when submitting a peer review to provide evidence or supporting materials. Teachers can attach conclusion documents or summary files to an assignment after the review period. Files are stored server-side and downloadable by the intended recipients.

Technical Approach

Reuse the file upload infrastructure built in Feature A (file_controller.py pattern). Create a new ReviewFile model linking uploaded files to specific reviews. Add a ConclusionFile model linking teacher uploads to assignments. Both models store filename, path, upload date, and uploader ID. The frontend adds file upload components to the review submission form and a new teacher conclusion section on the assignment detail page.

Accepted file types: PDF (.pdf), Images (.png, .jpg, .jpeg), Word documents (.docx). Maximum file size: 10MB per file.

Task Assignments

Dev	Role	Tasks	Branch
Dev 1	Backend: Review File Endpoints	Add POST /review/<id>/upload and GET /review/<id>/files endpoints. Students upload files during review submission. Validate file types (PDF, PNG, JPG, DOCX), enforce 10MB limit. Create download endpoint GET /review/file/<file_id>.	feature/review-files-endpoint
Dev 2	Backend: Models & Conclusion Endpoints	Create ReviewFile model (id, review_id, filename, path, uploaded_at, uploader_id) and ConclusionFile model (id, assignment_id, filename, path, uploaded_at, teacher_id). Add POST /assignment/<id>/conclusion/upload and GET /assignment/<id>/conclusion/files for	feature/review-files-endpoint

		teacher conclusions. Create Marshmallow schemas for both models.	
Dev 3	Backend: Tests	Write pytest tests: (1) Student uploads file with review, (2) Invalid file type rejected, (3) File size limit enforced, (4) Download review file works, (5) Teacher uploads conclusion file, (6) Student cannot upload conclusion, (7) Non-authenticated upload rejected, (8) List files for a review.	test/review-files
Dev 4	Frontend: Student Review File Upload	Add file upload component to the peer review submission form. Allow multiple file selection. Display attached files with remove option before submission. Add upload API calls to api.ts. Show attached files on the feedback viewing page (US12) as download links.	feature/review-files-ui
Dev 5	Frontend: Teacher Conclusion Section	Create ConclusionSection.tsx component for the teacher assignment detail page. Teacher can upload conclusion files after review period. Display uploaded conclusion files with download links visible to students. Handle loading, error, empty states.	feature/review-files-ui
Dev 6	Docs & Integration	Update ENDPOINT_SUMMARY.md with all new endpoints. Update user_stories.md for US1 and US12 extensions. Code review all PRs. Integration testing: student submits review with file, teacher uploads conclusion, both can download. Verify role-based access.	docs/review-files

API Endpoints

Method	Endpoint	Description	Auth
--------	----------	-------------	------

POST	/review/<id>/upload	Upload file(s) with review submission	@jwt_required() - Student
GET	/review/<id>/files	List files attached to a review	@jwt_required() - Any role
GET	/review/file/<file_id>	Download a specific review file	@jwt_required() - Any role
POST	/assignment/<id>/conclusion/upload	Teacher uploads conclusion file	@jwt_required() - Teacher
GET	/assignment/<id>/conclusion/files	List conclusion files for assignment	@jwt_required() - Any role

Example Backend Response

POST /review/3/upload (multipart/form-data):

```
{ "message": "File uploaded successfully", "file_id": 12, "filename": "evidence.pdf", "size": "1.8MB" }
```

GET /review/3/files:

```
{ "review_id": 3, "files": [{ "file_id": 12, "filename": "evidence.pdf", "uploaded_at": "2026-03-05T14:30:00Z", "size": "1.8MB" }] }
```

GET /assignment/5/conclusion/files:

```
{ "assignment_id": 5, "files": [{ "file_id": 15, "filename": "final_grades_summary.pdf", "uploaded_at": "2026-03-06T10:00:00Z", "teacher": "Dr. Smith" }] }
```

Branch Strategy

dev (base branch — must include Feature A backend first)

└─ feature/review-files-endpoint (Dev 1 + Dev 2)

└─ test/review-files (Dev 3)

└─ feature/review-files-ui (Dev 4 + Dev 5)

└─ docs/review-files (Dev 6)

Merge order: backend endpoint → tests → frontend UI → docs. All PRs target dev, use squash and merge.

Important: Branch from dev AFTER Feature A's backend has been merged, so the file upload infrastructure is available.

Sprint Schedule

Day 1-2: Setup & Backend Core

Dev	Tasks
Dev 1	Stub review file upload/download endpoints. Reuse file validation logic from Feature A's file_controller.py.
Dev 2	Create ReviewFile and ConclusionFile SQLAlchemy models. Create Marshmallow schemas. Add conclusion upload/list endpoints.
Dev 3	Set up test_review_files.py, write test fixtures. Create helper to seed review data for tests.
Dev 4	Study existing review submission form. Plan file upload integration points.

Dev 5	Study assignment detail page. Design ConclusionSection component layout.
Dev 6	Draft endpoint documentation for all 5 new endpoints.

Day 3-4: Core Implementation

Dev	Tasks
Dev 1+2	Complete all backend endpoints. Handle edge cases (no file, wrong type, role checks). Open backend PR.
Dev 3	Write all 8 test cases. Run coverage report.
Dev 4+5	Build file upload into review form (Dev 4) and ConclusionSection.tsx (Dev 5). Connect to real APIs. Open frontend PR.
Dev 6	Begin code reviews. Test role-based access (student vs teacher uploads).

Day 5-6: Testing & Polish

Dev	Tasks
Dev 1+2	Address review feedback. Fix bugs. Ensure proper role-based access control.
Dev 3	Add edge case tests for role permissions and concurrent uploads.

Dev 4+5	Polish UI: multiple file selection, remove before submit, download links on feedback page. Responsive design.
Dev 6	Finalize documentation. End-to-end integration testing.

Day 7: Integration & Merge

All team members: final integration testing. Dev 6 merges approved PRs to dev (squash and merge). Verify feature works end-to-end on dev branch.

Day 8: Buffer & Demo Prep

Fix any last-minute issues. Prepare demo for Thursday presentation.

Definition of Done

Code Quality

- ☐ Code follows PEP 8 (backend) and TypeScript conventions (frontend)
- ☐ No linting errors
- ☐ No commented-out code or debug statements
- ☐ Meaningful variable and function names

Testing

- ☐ Unit tests written for all new backend code
- ☐ All tests pass locally (pytest)
- ☐ All tests pass in CI/CD
- ☐ Edge cases tested

Documentation

- ☐ ENDPOINT_SUMMARY.md updated with new endpoints
- ☐ User story statuses updated in user_stories.md
- ☐ Code has docstrings explaining complex logic

Functionality

- ☐ Students can attach files (PDF, PNG, JPG, DOCX) when submitting peer reviews
- ☐ Teachers can upload conclusion files to assignments
- ☐ Attached files display as download links on feedback viewing page (US12)
- ☐ Conclusion files visible to students on assignment detail page
- ☐ Role-based access enforced (students cannot upload conclusions, etc.)
- ☐ Works on desktop and mobile viewports

Review & Merge

- ☐ Pull request created with clear description
- ☐ At least 1 peer review approval per PR
- ☐ All reviewer feedback addressed
- ☐ CI/CD pipeline passes
- ☐ Branches merged to dev and deleted

Risks & Mitigations

Risk	Mitigation
Feature A backend not merged in time	Dev 1+2 can build independently and refactor to share code later. Use same patterns.
Multiple file uploads create complexity	Start with single file upload, extend to multiple once stable.
Role-based access bugs (student uploading conclusions)	Dev 3 writes explicit role permission tests. Dev 6 tests manually.
Frontend blocked waiting for backend API	Dev 4+5 use mock data first, swap to real API after backend merges.
File storage conflicts with Feature A uploads	Use separate subdirectories: uploads/assignments/ and uploads/reviews/ and uploads/conclusions/.

New Endpoint Checklist (from dev-ops.md)

- ☐ Backend: Use `@jwt_required()` decorator on all new endpoints
- ☐ Frontend: Include credentials: 'include' in fetch options
- ☐ Frontend: No Authorization headers (cookies handle auth)
- ☐ Tests: Use `test_client` fixtures (handles cookies automatically)
- ☐ CORS: No changes needed (same localhost setup)
- ☐ Backend: File upload endpoints use `request.files` (Flask)
- ☐ Backend: Validate file types server-side (never trust client)
- ☐ Frontend: Use `multipart/form-data` for file upload requests